

Longest Common Subsequence (LCS)

1. what is LCS
2. LCS using Recursion
3. LCS using Memoization
4. LCS using Dynamic Programming



Longest Common Subsequence (LCS)

1. what is LCS
2. LCS using Recursion
3. LCS using Memoization
4. LCS using Dynamic Programming



Longest Common Subsequence (LCS)

String₁: a b c d e f g h i j
String₂: e c d g i

egi
cdgi



Longest Common Subsequence (LCS)

String₁: a b d a c e
String₂: b a b c e

b a c e



Longest Common Subsequence (LCS)

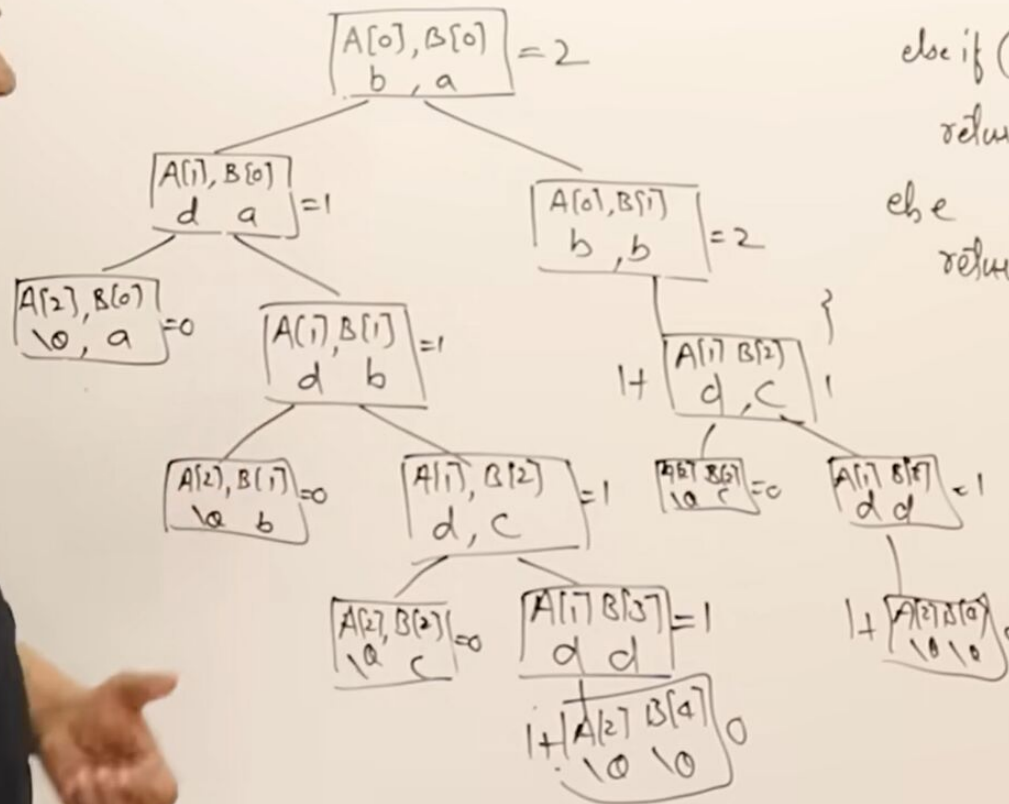
String1: a b d a c e
String2: b a b c e

bace
abce

Longest Common Subsequence (LCS)

A | b | d | \0
0 1 2

B | a | b | c | d | \0
0 1 2 3 4



```
int LCS(i, j)
{
```

```
    if (A[i] == '\0' || B[j] == '\0')
        return 0;
```

```
    else if (A[i] == B[j])
```

```
        return 1 + LCS(i+1, j+1);
```

```
    else
```

```
        return max(LCS(i+1, j), LCS(i, j+1));
```

Longest Common Subsequence (LCS)

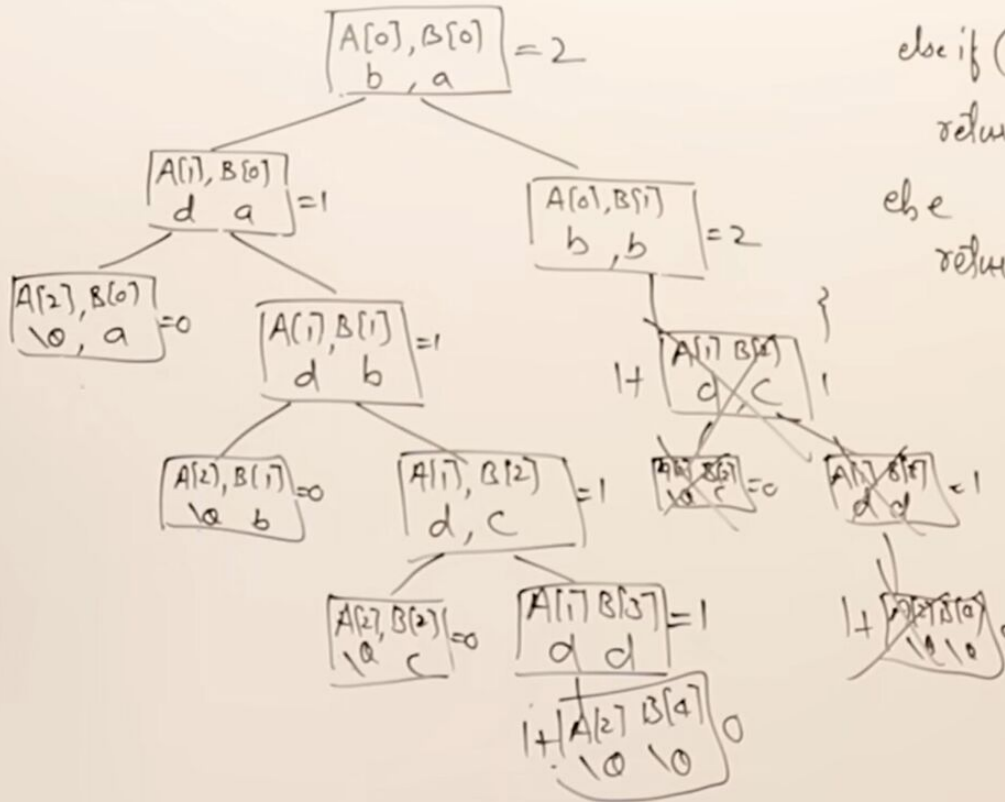
	a	b	c	d	∅
0	1	2	3	4	
b 0	2	2			
d 1	1	1	1	1	
∅ 2	0	0	0		0

A

b	d	∅
0	1	2

B

a	b	c	d	∅
0	1	2	3	4



```
int LCS(i, j)
{
```

```
    if (A[i] == '∅' || B[j] == '∅')
        return 0;
```

```
    else if (A[i] == B[j])
        return 1 + LCS(i+1, j+1);
```

```
    else
        return max(LCS(i+1, j), LCS(i, j+1));
```



Longest Common Subsequence (LCS)

	a	b	c	d	∅
0	0	1	2	3	4
b 0	2	2			
d 1	1	1	1	1	
∅ 2	0	0	0		0

$O(m \times n)$

A

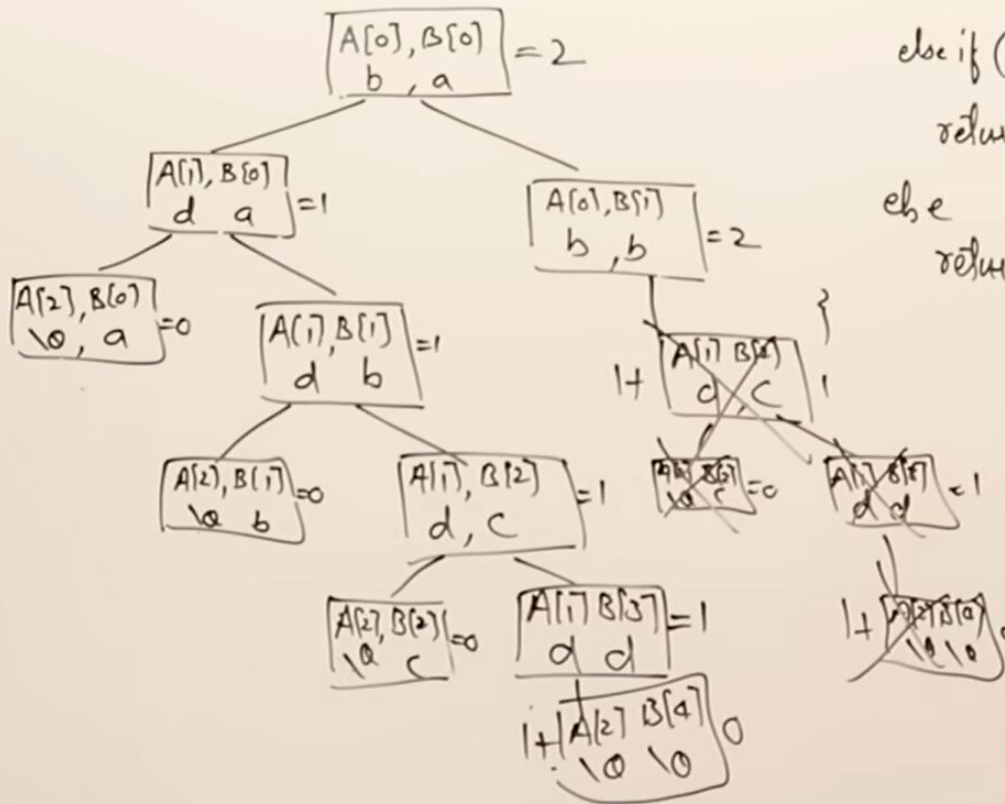
b	d	∅
0	1	2

 m

B

a	b	c	d	∅
0	1	2	3	4

 n



```
int LCS(i, j)
{
```

```
    if (A[i] == '∅' || B[j] == '∅')
        return 0;
```

```
    else if (A[i] == B[j])
        return 1 + LCS(i+1, j+1);
```

```
    else
        return max(LCS(i+1, j), LCS(i, j+1));
```



Longest Common Subsequence (LCS)

m A b d
1 2

n B a b c d
1 2 3 4

		a	b	c	d
	0	1	2	3	4
0	0	0	0	0	0
b 1	0	0	1	1	1
d 2	0	0	1	1	2

bd b d

if ($A[i] = B[j]$)

$$LCS[i, j] = 1 + LCS[i-1, j-1]$$

else

$$LCS[i, j] = \max(LCS[i-1, j], LCS[i, j-1])$$

$O(m \times n)$